

Disk Image Metadata & Hashing Analyzer Report

1. Introduction (Purpose & Problem)

Digital forensics is really important when it comes to investigating cyber incidents, especially when you need to make sure digital evidence hasn't been changed or tampered with. One of the biggest challenges investigators face is going through large amounts of files and verifying that everything is still intact. Doing this manually can take a lot of time and can also lead to mistakes.

The purpose of this project is to create a Disk Image Metadata & Hashing Analyzer, which is a Python-based tool that helps automate this process. Instead of manually checking files, the program collects important metadata and generates cryptographic hashes to verify file integrity. This makes the process faster, more accurate, and much easier to manage.

This tool helps solve a few common problems in digital forensics. It reduces the amount of manual work required, improves accuracy when verifying evidence, and helps with proper documentation for things like chain-of-custody. By collecting details like file names, paths, sizes, and timestamps, it gives investigators a clearer picture of what's going on with the files.

On top of that, the project includes an AI-based anomaly detection feature. This allows the system to look for unusual patterns in the metadata that could point to suspicious or tampered files. Adding this extra layer makes the tool more useful because it doesn't just collect data—it actually helps highlight what might be important.

2. Technical Implementation

This project was built using Python and is organized into four main parts: metadata extraction, hashing, exporting results, and anomaly detection. Breaking the program into these phases made it easier to manage and debug, since each part has a clear role.

Python was a good choice for this project because it's easy to read and has a lot of useful libraries. The `os` module is used to go through folders and files, while `pathlib` helps manage file paths in a cleaner way. The `hashlib` library is used to generate MD5, SHA-1, and SHA-256 hashes, which are important for verifying file integrity. The `datetime` module is used to handle timestamps, and `pandas` helps organize all the collected data into a structured format. For the AI part, the project uses `scikit-learn`, specifically the Isolation Forest algorithm.

In the first phase, the program scans through a folder and collects metadata for each file. This includes things like file name, path, type, size, and timestamps for when the file was created, modified, and accessed. It also collects additional details like the depth of the file path, which

can help with analysis later. This phase is important because metadata gives context about how files are being used.

The second phase focuses on hashing. For each file, the program generates MD5, SHA-1, and SHA-256 hashes. These hashes act like digital fingerprints, meaning even a small change in a file will result in a completely different hash. This is useful for confirming that files haven't been altered.

In the third phase, all the collected data is exported into CSV and JSON files. This makes it easy to view the results in spreadsheets or use them in other tools. It also helps with documentation, which is important in forensic investigations.

The final phase is where the AI comes in. The program uses Isolation Forest to analyze the metadata and detect unusual patterns. It looks at features like file size, timestamps, and file types, then assigns an anomaly score to each file. Files that stand out as unusual are labeled as suspicious, while the rest are marked as normal.

3. Results

The tool produces a dataset that includes file metadata, hash values, and anomaly detection results. Each file is listed with its details, along with whether it was flagged as normal or suspicious. This makes it easy to quickly scan through the results and identify anything that might need further investigation.

The results are saved as both CSV and JSON files, which makes them flexible and easy to use. Investigators can open the CSV file in a spreadsheet or use the JSON file in other programs. This also helps with keeping proper records and documenting the investigation process.

From testing the tool, most files are labeled as normal, while a small number are marked as suspicious. These flagged files usually have unusual sizes, strange timestamps, or uncommon file types. This helps narrow down the search and allows investigators to focus on files that might actually be important.

Overall, the tool does a good job of automating tasks that would normally take a long time to do manually. It also adds extra value by highlighting potential anomalies instead of just listing data.

4. Lessons Learned & Conclusion

Working on this project helped reinforce a lot of important concepts in both cybersecurity and digital forensics. One thing that worked really well was organizing the code into separate phases. This made everything easier to understand and fix when something went wrong. Python libraries like pandas and scikit-learn also made it much easier to handle data and implement machine learning features.

At the same time, there were a few challenges. Dealing with file access issues, like permission errors or corrupted files, required extra error handling. Choosing the right features for the anomaly detection model was also tricky, since the model's performance depends on the quality of the data. Understanding how to interpret anomaly scores and decide what counts as "suspicious" also took some trial and error.

If this project were to be improved in the future, there are a few things that could be added. For example, support for actual disk image formats like `.dd` or `.E01` would make it more useful in real-world investigations. Adding deeper file content analysis could also improve accuracy. A graphical interface would make the tool easier to use, especially for people who aren't comfortable with command-line tools.

In conclusion, the Disk Image Metadata & Hashing Analyzer successfully demonstrates how automation and machine learning can be used in digital forensics. It helps verify file integrity, organize evidence, and identify unusual patterns in data. Overall, this project shows how combining traditional forensic methods with AI can make investigations more efficient and effective.